BANK CUSTOMER CHURN PREDICTION

The objective of this project is:

To analyze customer data and identify patterns that lead to customer churn (customers leaving the bank), and build insights that help the bank reduce churn.

Business Goal: Identify why customers leave Find high-risk customers Help bank to improve retention strategy

In [44]:
```
import pandas as pd               #  data handling (DataFrame)
import numpy as np                 # numerical operations
import matplotlib.pyplot as plt    # this two libraries r for visvalization so whatever the charts we will make we  w
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')      #  warning we r importing bcz whenever we r doing coding sometimes a red line
%matplotlib inline
```

In [45]:
```
pwd
```

Out[45]:
```
'C:\\Users\\kchau'
```

Loads dataset

In [46]:
```
data = pd.read_csv("D:\Data Anyaletics/Bank Customer Churn Prediction.csv")
data
```

Out[46]:

	customer_id	credit_score	country	gender	age	tenure	balance	products_number	credit_card	active_member	estim
0	15634602	619	France	Female	42	2	0.00	1	1	1	
1	15647311	608	Spain	Female	41	1	83807.86	1	0	1	
2	15619304	502	France	Female	42	8	159660.80	3	1	0	
3	15701354	699	France	Female	39	1	0.00	2	0	0	
4	15737888	850	Spain	Female	43	2	125510.82	1	1	1	
...	...	...	...	...	...	...	...	...	...	...	
9995	15606229	771	France	Male	39	5	0.00	2	1	0	
9996	15569892	516	France	Male	35	10	57369.61	1	1	1	
9997	15584532	709	France	Female	36	7	0.00	1	0	1	
9998	15682355	772	Germany	Male	42	3	75075.31	2	1	0	
9999	15628319	792	France	Female	28	4	130142.79	1	1	0	

10000 rows × 12 columns

BASIC DATA INSPECTION

It Shows all column names

In [47]:
```
data.columns
```

Out[47]:
```
Index(['customer_id', 'credit_score', 'country', 'gender', 'age', 'tenure',
       'balance', 'products_number', 'credit_card', 'active_member',
       'estimated_salary', 'churn'],
      dtype='object')
```

Rows = number of customers Columns = number of features rows are 10000 and columns are 12

In [48]:
```
data.shape
```

Out[48]: (10000, 12)

data.info function helps to understand the data types ,null values and memory usage .so dtypes are int,float and object ,their is no null values menas it indicates that the dataset is clean and does not require imputation techniques like mean, median, or mode filling. and memory usage is 97.6+kb

In [49]:
```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 12 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   customer_id       10000 non-null  int64
 1   credit_score      10000 non-null  int64
 2   country           10000 non-null  object
 3   gender            10000 non-null  object
 4   age               10000 non-null  int64
 5   tenure            10000 non-null  int64
 6   balance           10000 non-null  float64
 7   products_number   10000 non-null  int64
 8   credit_card       10000 non-null  int64
 9   active_member     10000 non-null  int64
 10  estimated_salary  10000 non-null  float64
 11  churn             10000 non-null  int64
dtypes: float64(2), int64(8), object(2)
memory usage: 937.6+ KB
```

This data.dtypes function I checked to understand the structure of the dataset.

Most of the columns like credit_score, age, tenure, and products_number are integer types (int64), which represent discrete numerical values. Columns like balance and estimated_salary are float64, indicating continuous numerical data with decimal values. country and gender are object type, which means they are categorical variables stored as strings. The target variable churn is also an integer (0 and 1), representing a binary classification problem.

This step is important because it helps decide:

What preprocessing is required Which columns need encoding And how to apply statistical analysis or machine learning models.

note : No major issues. Data types are mostly appropriate. Only categorical columns (country, gender) require encoding before model building.

In [50]:
```
data.dtypes
```

Out[50]:
```
customer_id           int64
credit_score          int64
country              object
gender               object
age                   int64
tenure                int64
balance             float64
products_number       int64
credit_card           int64
active_member         int64
estimated_salary    float64
churn                 int64
dtype: object
```

I used data.describe() to understand the statistical distribution of numerical features in the dataset.

The dataset contains 10,000 records with no missing values in numerical columns.

The average credit score is around 650, and the average age is approximately 39 years, indicating a middle-aged customer base.

The average balance is around 76,000, but since the minimum balance is 0 and the median is around 97,000, it suggests that a significant number of customers have zero balance.

The churn rate is about 20%, meaning roughly 1 in 5 customers have left the bank.

Overall, the dataset shows variability in financial and demographic features, which is useful for predictive modeling."

In [51]:
```
data.describe()
```

Out[51]:

	customer_id	credit_score	age	tenure	balance	products_number	credit_card	active_member
count	1.000000e+04	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000	10000.00000	10000.000000
mean	1.569094e+07	650.528800	38.921800	5.012800	76485.889288	1.530200	0.70550	0.515100
std	7.193619e+04	96.653299	10.487806	2.892174	62397.405202	0.581654	0.45584	0.499797
min	1.556570e+07	350.000000	18.000000	0.000000	0.000000	1.000000	0.00000	0.000000
25%	1.562853e+07	584.000000	32.000000	3.000000	0.000000	1.000000	0.00000	0.000000
50%	1.569074e+07	652.000000	37.000000	5.000000	97198.540000	1.000000	1.00000	1.000000
75%	1.575323e+07	718.000000	44.000000	7.000000	127644.240000	2.000000	1.00000	1.000000
max	1.581569e+07	850.000000	92.000000	10.000000	250898.090000	4.000000	1.00000	1.000000

In [52]:
```
data.describe().T
```

Out[52]:

	count	mean	std	min	25%	50%	75%	max
customer_id	10000.0	1.569094e+07	71936.186123	15565701.00	15628528.25	1.569074e+07	1.575323e+07	15815690.00
credit_score	10000.0	6.505288e+02	96.653299	350.00	584.00	6.520000e+02	7.180000e+02	850.00
age	10000.0	3.892180e+01	10.487806	18.00	32.00	3.700000e+01	4.400000e+01	92.00
tenure	10000.0	5.012800e+00	2.892174	0.00	3.00	5.000000e+00	7.000000e+00	10.00
balance	10000.0	7.648589e+04	62397.405202	0.00	0.00	9.719854e+04	1.276442e+05	250898.09
products_number	10000.0	1.530200e+00	0.581654	1.00	1.00	1.000000e+00	2.000000e+00	4.00
credit_card	10000.0	7.055000e-01	0.455840	0.00	0.00	1.000000e+00	1.000000e+00	1.00
active_member	10000.0	5.151000e-01	0.499797	0.00	0.00	1.000000e+00	1.000000e+00	1.00
estimated_salary	10000.0	1.000902e+05	57510.492818	11.58	51002.11	1.001939e+05	1.493882e+05	199992.48
churn	10000.0	2.037000e-01	0.402769	0.00	0.00	0.000000e+00	0.000000e+00	1.00

I checked for missing values using data.isnull().sum(), and found that there are no missing values in any column. This means the dataset is clean and does not require any imputation techniques like mean, median, or mode filling. Having no missing values helps ensure better model performance and simplifies preprocessing.

In [53]:
```
data.isnull().sum()
```

Out[53]:
```
customer_id         0
credit_score        0
country             0
gender              0
age                 0
tenure              0
balance             0
products_number     0
credit_card         0
active_member       0
estimated_salary    0
churn               0
dtype: int64
```

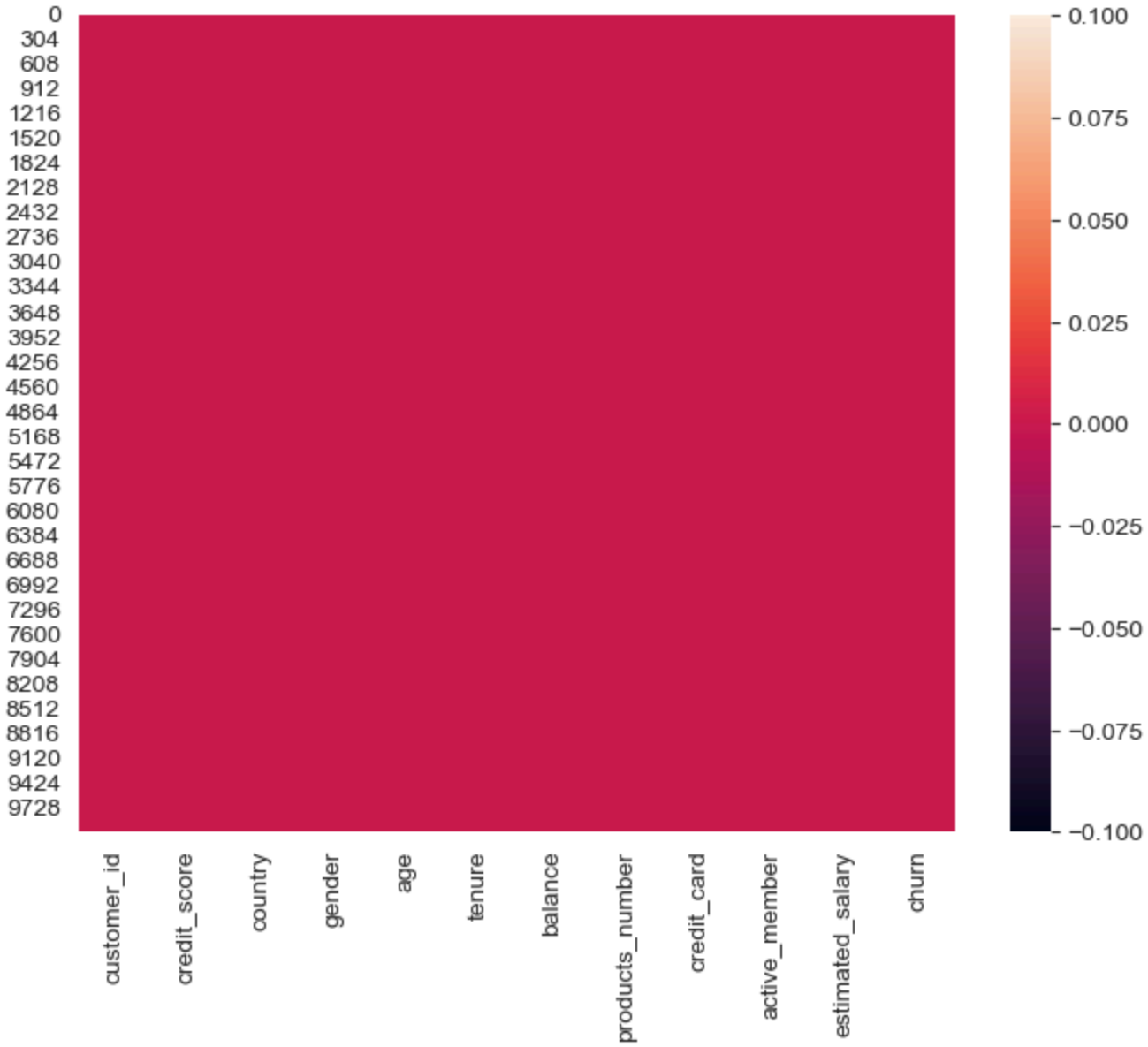
I checked for duplicate records using data.duplicated().sum(), and the result was 0, which means there are no duplicate rows in the dataset. This ensures data quality and prevents bias in analysis or model training. Duplicate records can distort analysis and lead to incorrect model predictions, so it's important to remove them if present.

In [54]:
```
data.duplicated().sum()
```

Out[54]:
```
np.int64(0)
```

In [55]:
```
plt.figure(figsize=(8,6))                          # i am using this graph to visualizes the missing values in the
sns.heatmap(data.isnull(),cbar=True)
```

Out[55]:
```
<Axes: >
```



I used data.nunique() to check the number of unique values in each column. This helps in understanding feature diversity, identifying categorical variables, and detecting columns that may not be useful for modeling.

For example, customer_id has 10,000 unique values, which means it is a unique identifier and not useful for prediction. Columns like country and gender have very few unique values, indicating categorical features. The target variable churn has 2 unique values, confirming it is a binary classification problem.

In [56]:
```
data.nunique()
```

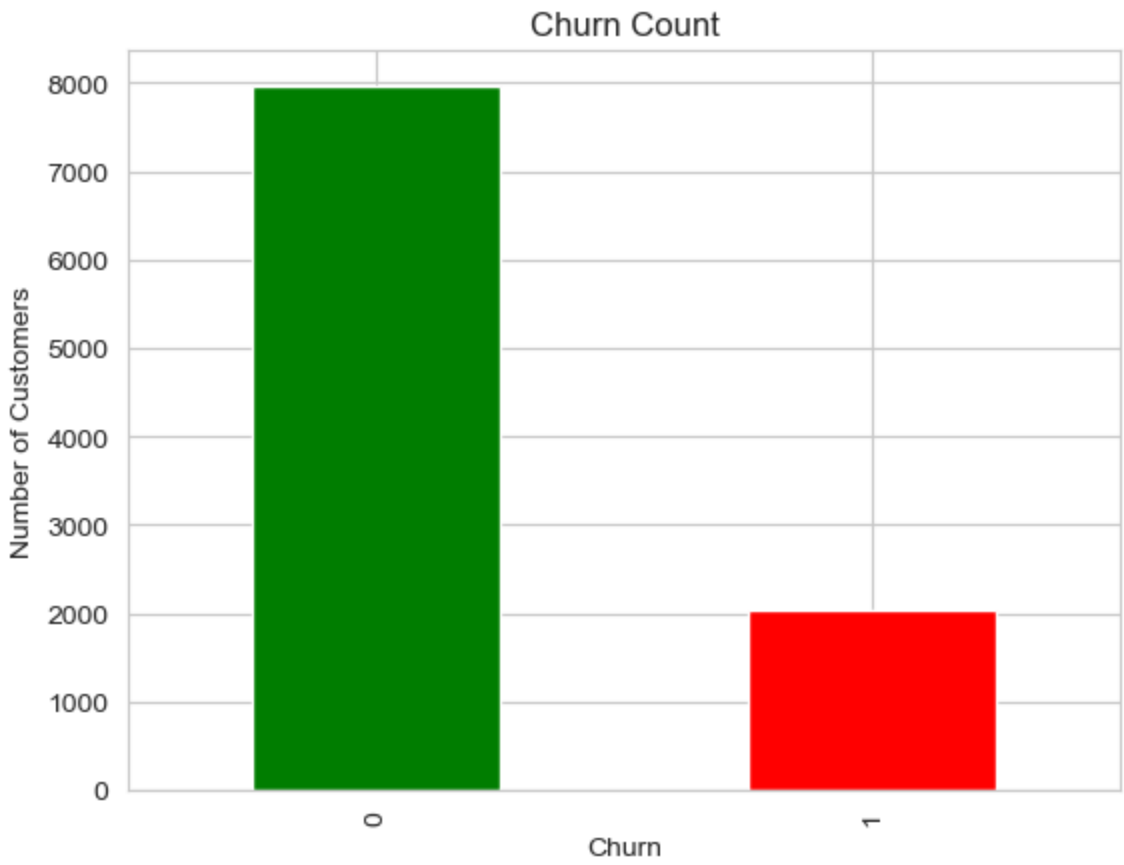
Out[56]:
```
customer_id         10000
credit_score          460
country                 3
gender                  2
age                    70
tenure                 11
balance              6382
products_number         4
credit_card             2
active_member           2
estimated_salary     9999
churn                   2
dtype: int64
```

Now START EDA (Exploratory Data Analysis)

Step 1: Target Analysis(Distribution of Target)

I plotted a bar chart of the target variable(churn) to understand its distribution. The graph shows that the number of non-churned customers is significantly higher than churned customers. This indicates a class imbalance in the dataset, where approximately 80% customers stayed and 20% churned.

In [57]:
```
data['churn'].value_counts().plot(
    kind='bar',
    color=['green', 'red']
)
plt.title("Churn Count")
plt.xlabel("Churn")
plt.ylabel("Number of Customers")
plt.show()
```



Percentage of Churn

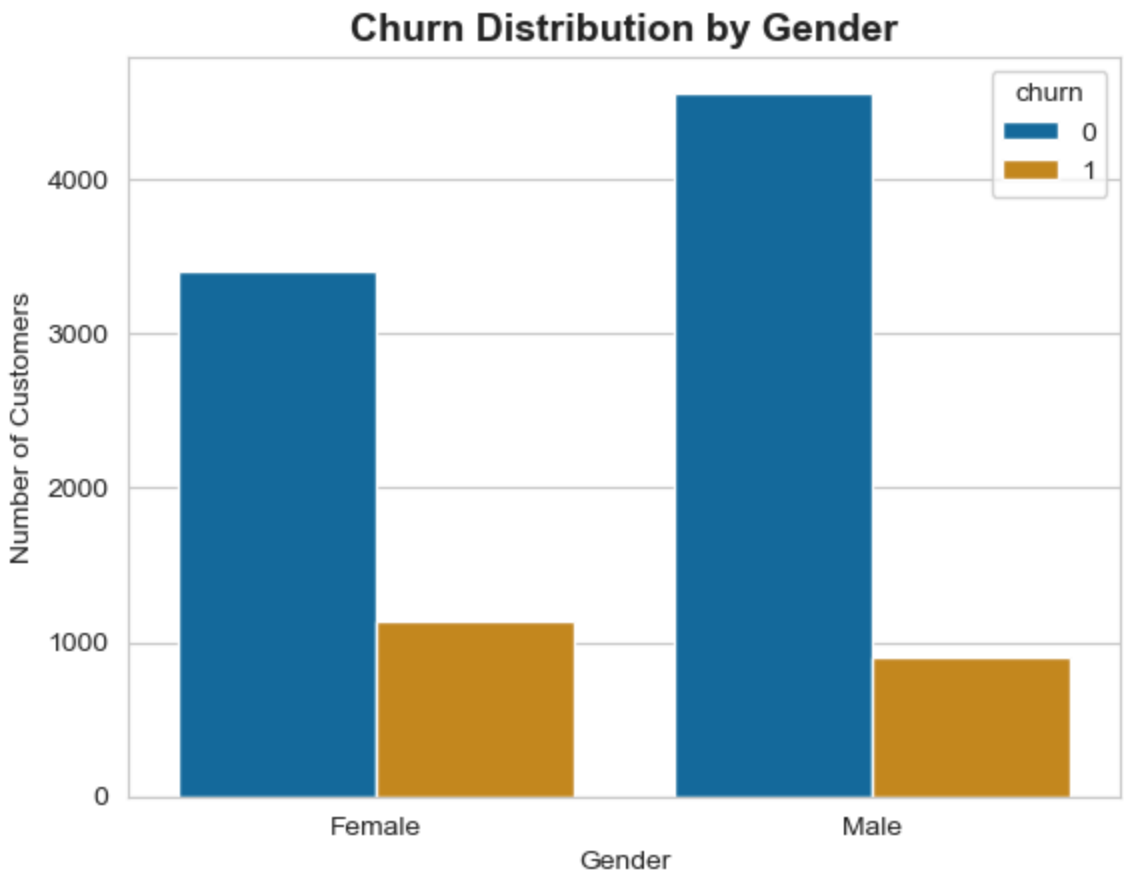
I calculated the percentage distribution of the target variable using normalized value counts. Around 79.63% customers did not churn, while 20.37% customers churned. This indicates a class imbalance in the dataset.

In [58]:
```
data['churn'].value_counts(normalize=True) * 100
```

Out[58]:
```
churn
0    79.63
1    20.37
Name: proportion, dtype: float64
```

Categorical vs Churn analysis

In [59]:
```
sns.set_style("whitegrid")

sns.countplot(
    x='gender',
    hue='churn',
    data=data,
    palette='colorblind'
)

plt.title("Churn Distribution by Gender", fontsize=14, fontweight='bold')
plt.xlabel("Gender")
plt.ylabel("Number of Customers")

plt.show()
```



OBSERVATIONS : I used a count plot to analyze the relationship between gender and churn. The plot shows the number of churned and non-churned customers for each gender. From the visualization, we can compare whether churn behavior differs between males and females.

NOTE :Female customers tend to have a slightly higher churn rate compared to male customers

In [60]:
```
data.groupby('gender')['churn'].value_counts(normalize=True).mul(100)
```

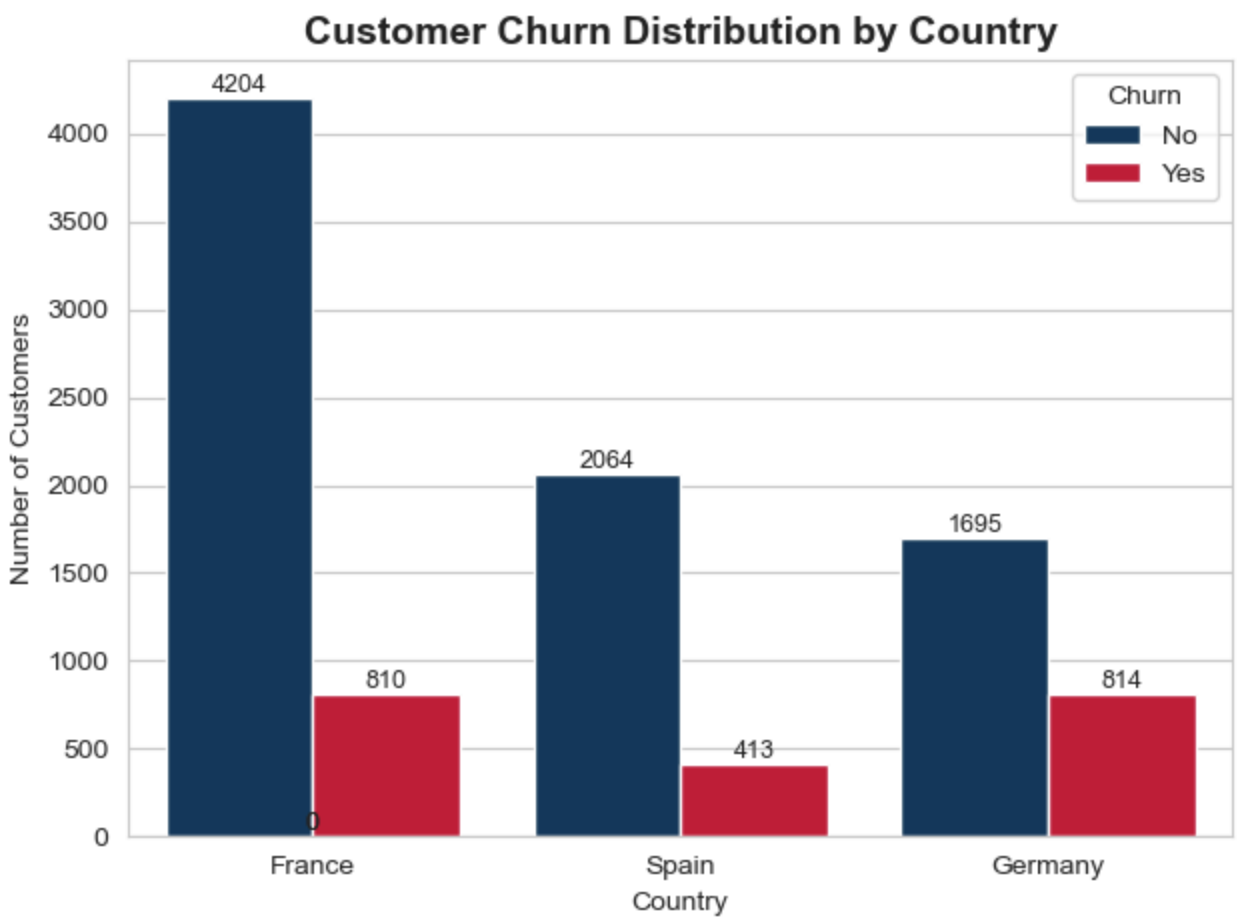
Out[60]:
```
gender  churn
Female  0        74.928461
        1        25.071539
Male    0        83.544072
        1        16.455928
Name: proportion, dtype: float64
```

OBSERVATION: I grouped the data by gender and calculated the percentage of churn within each group. This helped me understand the churn rate for male and female customers separately. I observed that female customers have a slightly higher churn percentage compared to male customers.

Customer Churn Distribution by Country

In [61]:
```
ax = sns.countplot(
    x='country',
    hue='churn',
    data=data,
    palette=['#0d3b66', '#d90429']   # dark blue & red
)

# Add labels on bars
for p in ax.patches:
    ax.annotate(f'{int(p.get_height())}',
                (p.get_x() + p.get_width()/2., p.get_height()),
                ha='center', va='bottom', fontsize=9)

plt.title("Customer Churn Distribution by Country", fontsize=14, fontweight='bold')
plt.xlabel("Country")
plt.ylabel("Number of Customers")

plt.legend(title="Churn", labels=["No", "Yes"])
```

```
plt.tight_layout()
plt.show()
```



OBSERVATION: I created a count plot to analyze churn distribution across different countries. The visualization shows the number of churned and non-churned customers for each country, with values displayed on top of each bar for better clarity. This helps identify geographical patterns in customer churn. NOTE: The plot shows that Germany has a higher churn rate compared to France and Spain.

In [62]:
```
pd.crosstab(data['country'], data['churn'], normalize='index') * 100
```

Out[62]:

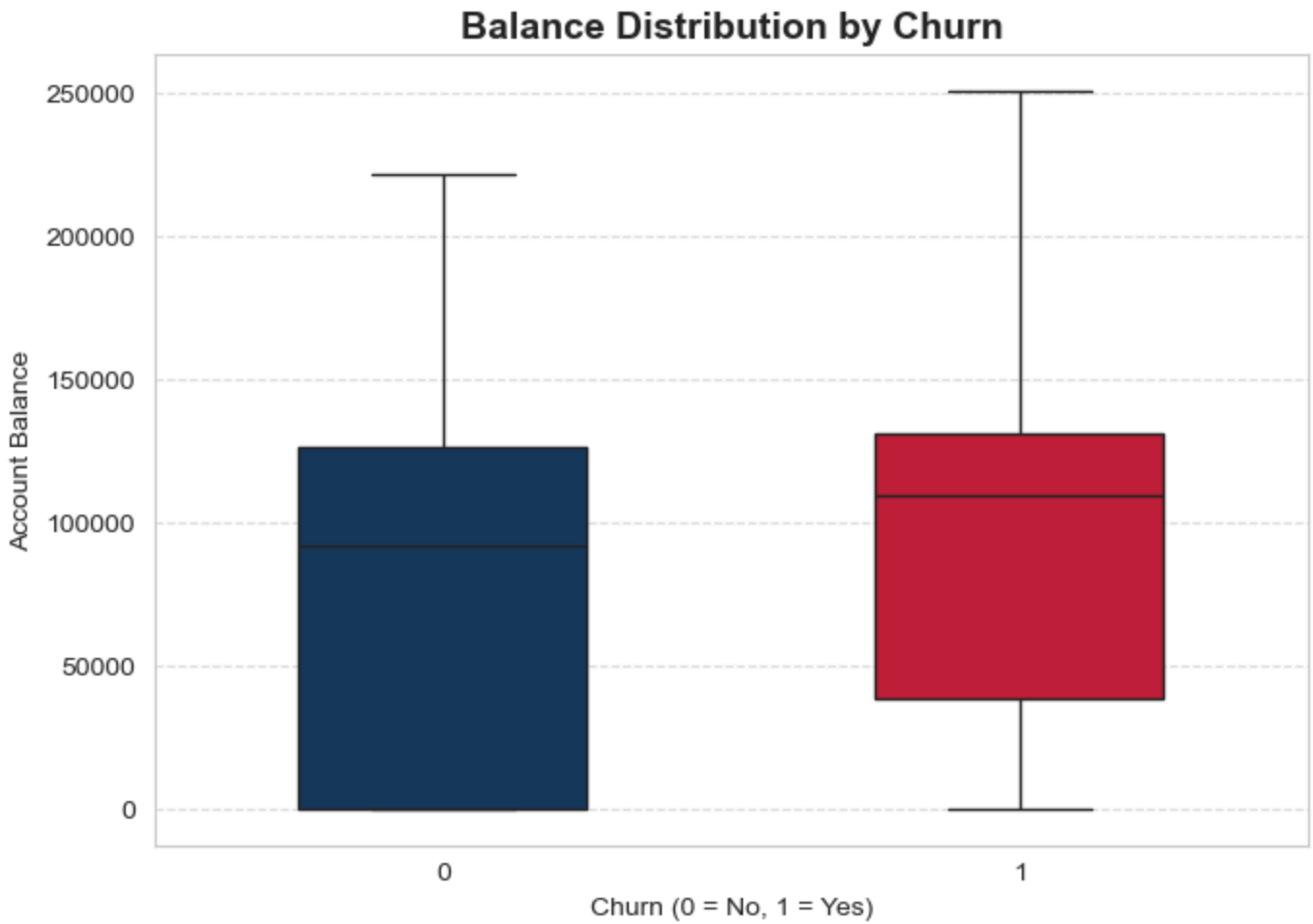
churn	0	1
country		
France	83.845233	16.154767
Germany	67.556796	32.443204
Spain	83.326605	16.673395

OBSERVATION: I used a cross-tabulation to calculate the percentage of churn within each country. This helps compare churn rates across countries independent of their population size. I observed that Germany has a higher churn rate compared to France and Spain.

Numerical vs Target analysis

In [63]:

```
sns.set_style("whitegrid")

plt.figure(figsize=(7,5))

sns.boxplot(
    x='churn',
    y='balance',
    data=data,
    palette=['#0d3b66', '#d90429'],
    width=0.5,
    fliersize=3
)

plt.title("Balance Distribution by Churn", fontsize=14, fontweight='bold')
plt.xlabel("Churn (0 = No, 1 = Yes)")
plt.ylabel("Account Balance")

plt.grid(axis='y', linestyle='--', alpha=0.6)
plt.tight_layout()
plt.show()
```



observation: I used a boxplot to compare the distribution of account balance between churned and non-churned customers. The plot shows that customers who churned generally have higher account balances compared to those who did not churn

In [64]:
```
pd.crosstab(pd.cut(data['balance'], bins=5), data['churn'], normalize='index') * 100
```

Out[64]:

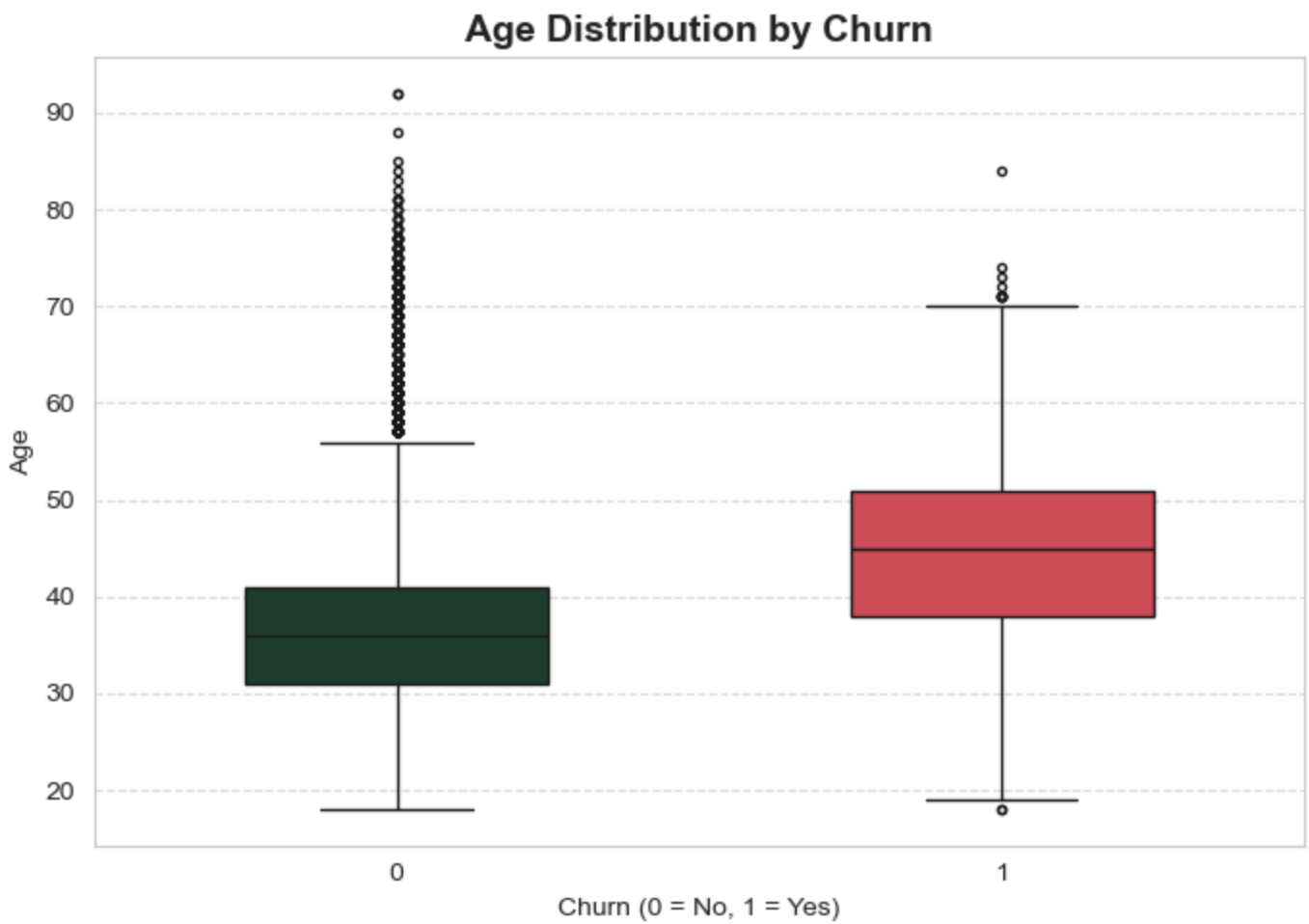
churn	0	1
balance		
(-250.898, 50179.618]	85.752979	14.247021
(50179.618, 100359.236]	80.039139	19.960861
(100359.236, 150538.854]	74.216301	25.783699
(150538.854, 200718.472]	78.251366	21.748634
(200718.472, 250898.09]	40.625000	59.375000

observation :I divided the balance into different ranges and checked how many customers churn in each range. I found that customers with higher balance are more likely to churn.

AGE DISTRIBUTION BY CHURN

In [65]:

```
plt.figure(figsize=(7,5))

sns.boxplot(
    x='churn',
    y='age',
    data=data,
    palette=['#1b4332', '#e63946'],   # dark green vs red
    width=0.5,
    fliersize=3
)

plt.title("Age Distribution by Churn", fontsize=14, fontweight='bold')
plt.xlabel("Churn (0 = No, 1 = Yes)")
plt.ylabel("Age")

plt.grid(axis='y', linestyle='--', alpha=0.6)
plt.tight_layout()
plt.show()
```



OBSERVATION:I used a boxplot to compare the age of customers who churned and those who didn't. I found that customers who churned are generally older

In [66]:
```
pd.crosstab(pd.cut(data['age'], bins=5), data['churn'], normalize='index') * 100
```

Out[66]:

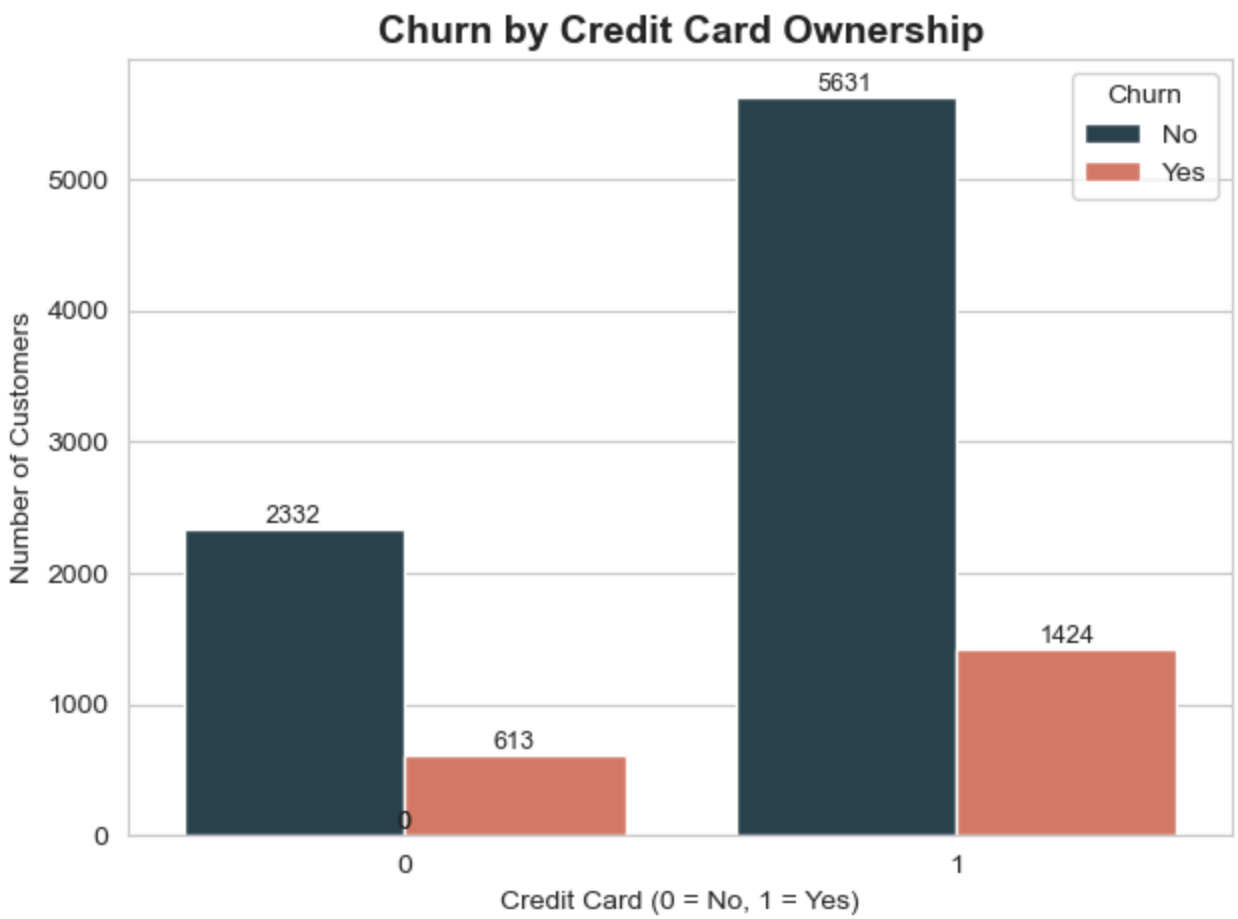
churn	0	1
age		
(17.926, 32.8]	92.365591	7.634409
(32.8, 47.6]	81.181818	18.818182
(47.6, 62.4]	47.002221	52.997779
(62.4, 77.2]	78.507463	21.492537
(77.2, 92.0]	95.833333	4.166667

OBSERVATION:I divided age into different groups and checked churn in each group. I found that older age groups have higher churn. Binning helps us see trends clearly, and here it shows a strong relationship between age and churn

Binary columns vs Target variable analysis

In [67]:

```
ax = sns.countplot(
    x='credit_card',
    hue='churn',
    data=data,
    palette=['#264653', '#e76f51']   # dark teal & orange-red
)

# Add labels
for p in ax.patches:
    ax.annotate(f'{int(p.get_height())}',
                (p.get_x() + p.get_width()/2., p.get_height()),
                ha='center', va='bottom', fontsize=9)

plt.title("Churn by Credit Card Ownership", fontsize=14, fontweight='bold')
plt.xlabel("Credit Card (0 = No, 1 = Yes)")
plt.ylabel("Number of Customers")

plt.legend(title="Churn", labels=["No", "Yes"])
plt.tight_layout()
plt.show()
```



OBSERVATION: Iused a count plot to compare churn between customers who have a credit card and those who don't. The result shows that there is no major difference in churn based on credit card ownership.

In [68]:
```
pd.crosstab(data['credit_card'], data['churn'], normalize='index') * 100
```

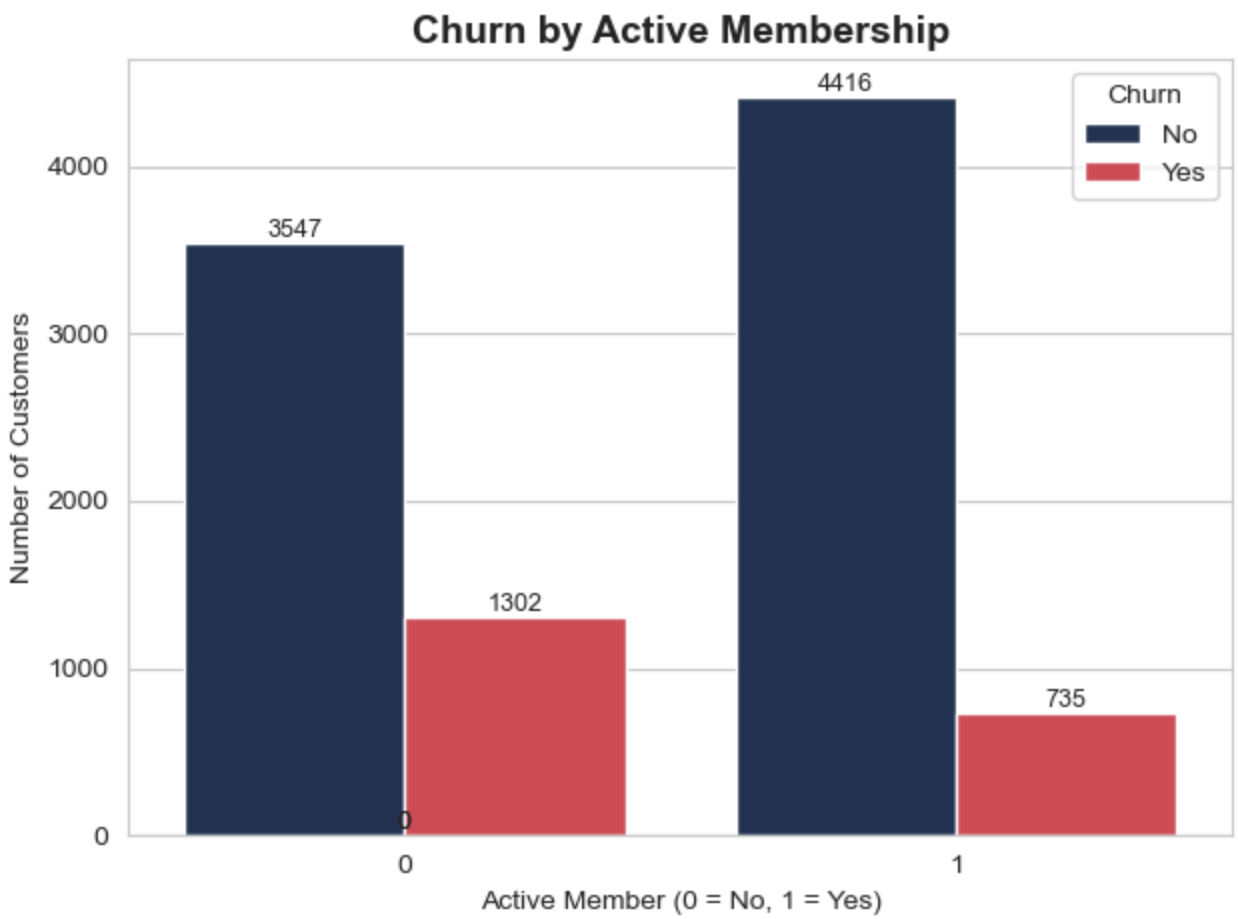
Out[68]:

churn	0	1
credit_card		
0	79.185059	20.814941
1	79.815734	20.184266

OBSERVATIONS:I used crosstab with normalization to compare churn percentage across credit card ownership. I found that both groups have almost similar churn rates, so credit card ownership does not significantly impact churn

Churn by Active Membership

In [69]:
```
ax = sns.countplot(
    x='active_member',
    hue='churn',
    data=data,
    palette=['#1d3557', '#e63946']   # dark blue & red
)

# Add labels
for p in ax.patches:
    ax.annotate(f'{int(p.get_height())}',
                (p.get_x() + p.get_width()/2., p.get_height()),
                ha='center', va='bottom', fontsize=9)

plt.title("Churn by Active Membership", fontsize=14, fontweight='bold')
plt.xlabel("Active Member (0 = No, 1 = Yes)")
plt.ylabel("Number of Customers")

plt.legend(title="Churn", labels=["No", "Yes"])
plt.tight_layout()
plt.show()
```



OBSERVATION:I analyzed churn based on active membership and found that inactive customers have a much higher churn rate compared to active customers. This indicates that customer engagement plays an important role in retention.

In [70]:
```
pd.crosstab(data['active_member'], data['churn'], normalize='index')*100
```

Out[70]:

churn	0	1
active_member		
0	73.149103	26.850897
1	85.730926	14.269074

OBSERVATION:Icompared churn percentage between active and inactive customers. I found that inactive customers have significantly higher churn (~27%) compared to active customers (~14%). This shows that active membership strongly impacts customer retention

In [71]:
```
# COLUMN TYPE SEGREGATION

cat_cols = ['country', 'gender']
bin_cols = ['credit_card', 'active_member', 'churn']
num_cols = ['credit_score', 'age', 'tenure', 'balance', 'products_number', 'estimated_salary']
```

OBSERVATIONS:I segregated the columns into categorical, binary, and numerical features to apply appropriate preprocessing techniques. Categorical columns like country and gender require encoding, binary columns are already in 0/1 format, and numerical columns can be used for scaling and modeling

In [72]:
```
cat_cols
```

Out[72]:
```
['country', 'gender']
```

In [73]:
```
bin_cols
```

Out[73]:
```
['credit_card', 'active_member', 'churn']
```

In [74]:
```
num_cols
```

Out[74]:
```
['credit_score',
 'age',
 'tenure',
 'balance',
 'products_number',
 'estimated_salary']
```

In [75]:
```
for col in cat_cols:
    print(col)
    print(data[col].value_counts())
```

```
country
country
France     5014
Germany    2509
Spain      2477
Name: count, dtype: int64
gender
gender
Male      5457
Female    4543
Name: count, dtype: int64
```

In [76]:
```
numeric_data = data[num_cols]
numeric_data
```

Out[76]:

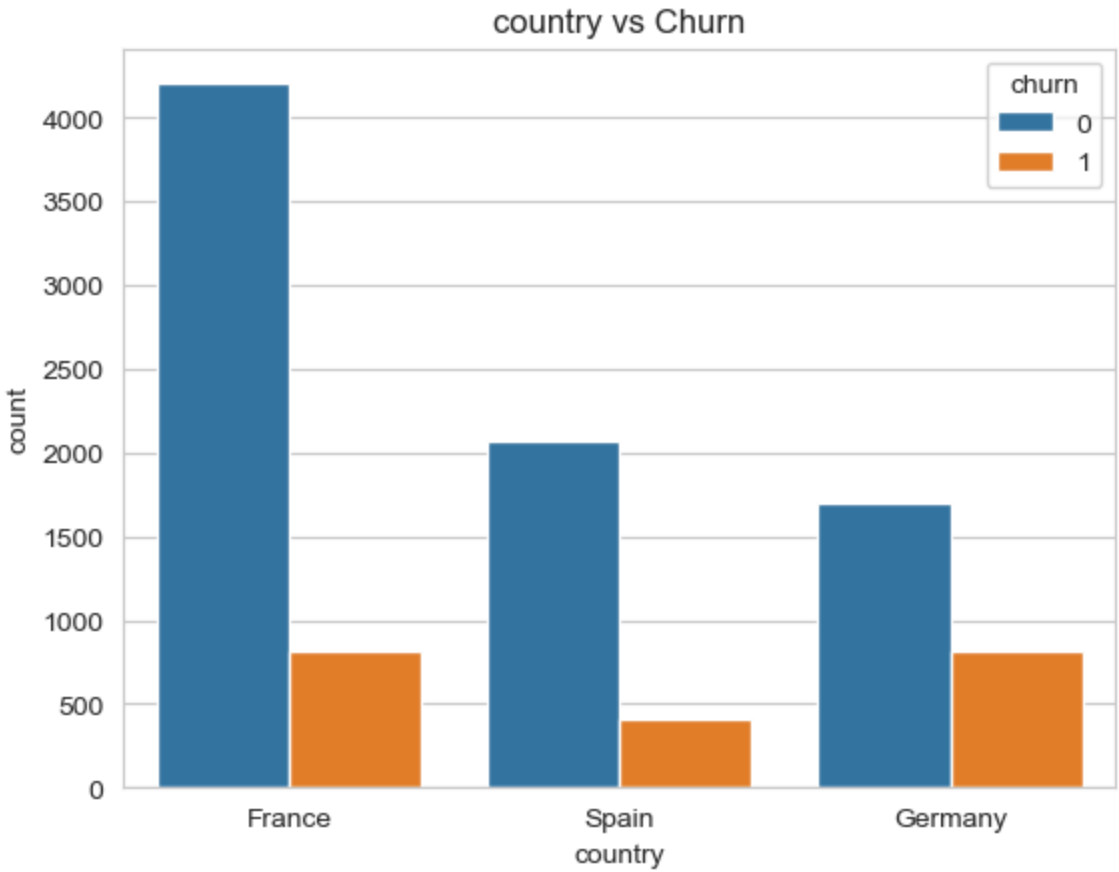
	credit_score	age	tenure	balance	products_number	estimated_salary
0	619	42	2	0.00	1	101348.88
1	608	41	1	83807.86	1	112542.58
2	502	42	8	159660.80	3	113931.57
3	699	39	1	0.00	2	93826.63
4	850	43	2	125510.82	1	79084.10
...	...	...	...	...	...	...
9995	771	39	5	0.00	2	96270.64
9996	516	35	10	57369.61	1	101699.77
9997	709	36	7	0.00	1	42085.58
9998	772	42	3	75075.31	2	92888.52
9999	792	28	4	130142.79	1	38190.78

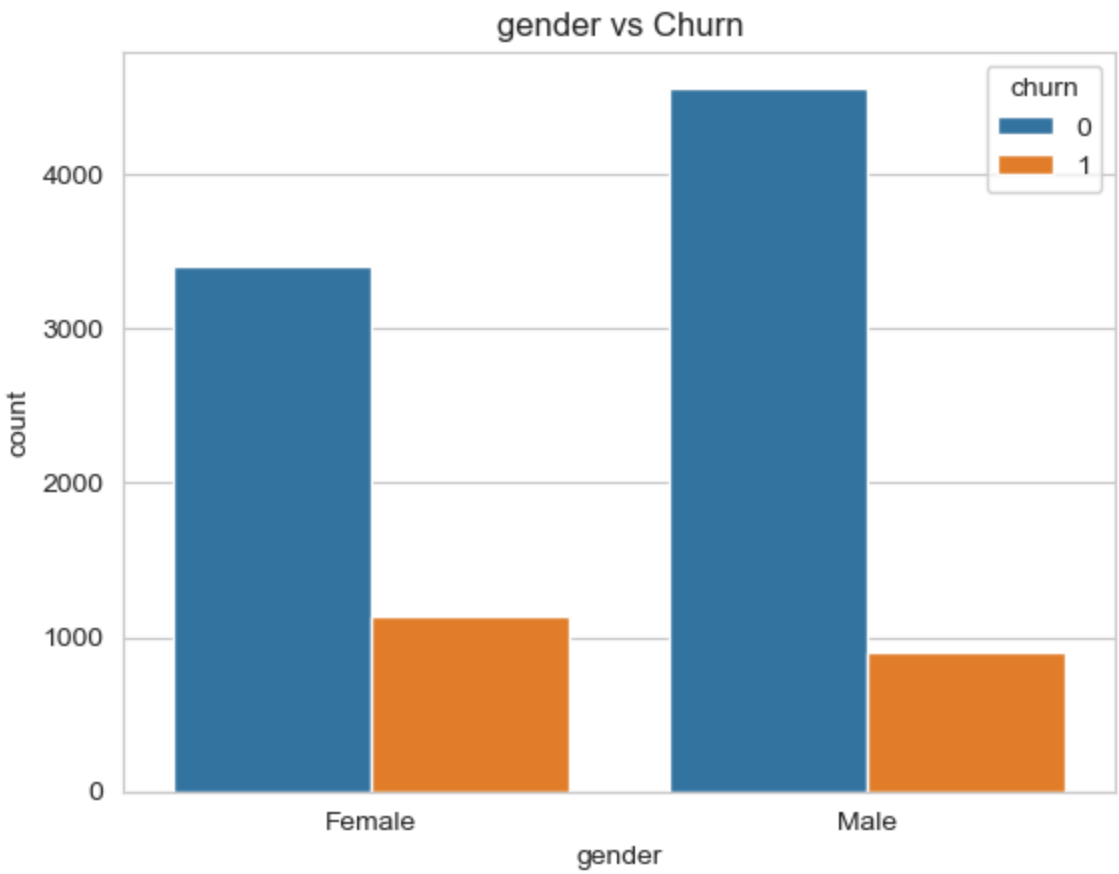
10000 rows × 6 columns

OBSERVATIONS:I printed value counts for categorical columns to understand the distribution of categories. This helps identify dominant categories and check for imbalance

Feature Segregation

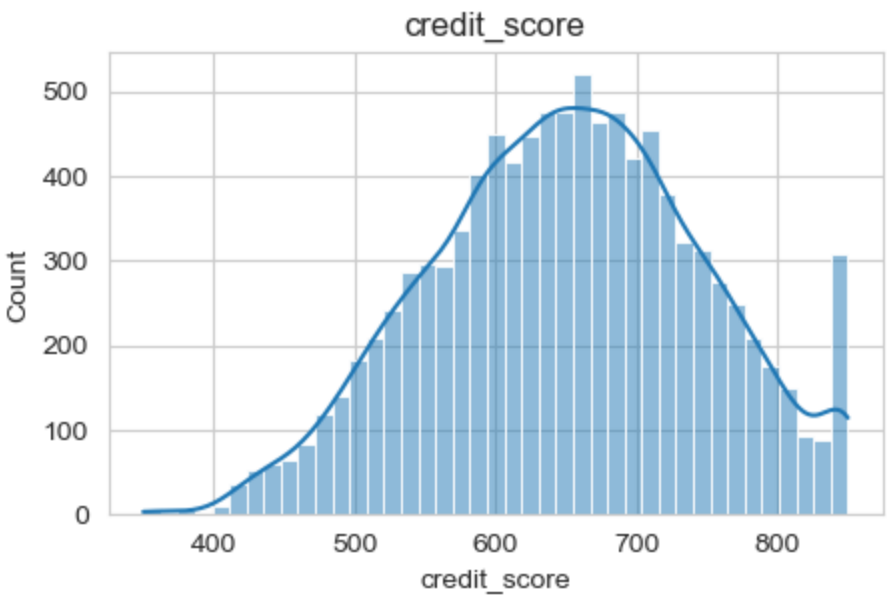
In [77]:
```
import seaborn as sns
import matplotlib.pyplot as plt

for col in cat_cols:
    sns.countplot(x=col, hue='churn', data=data)
    plt.title(f"{col} vs Churn")
    plt.show()
```

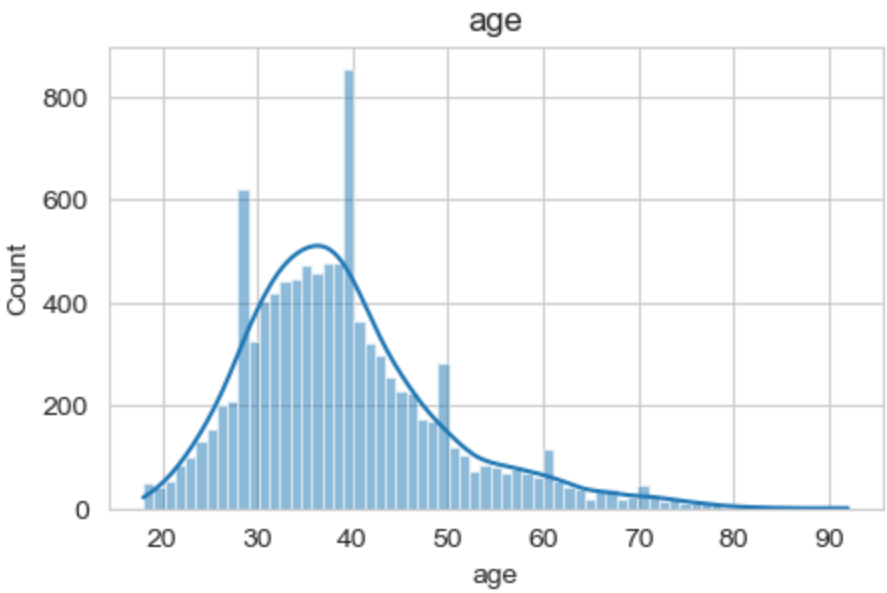


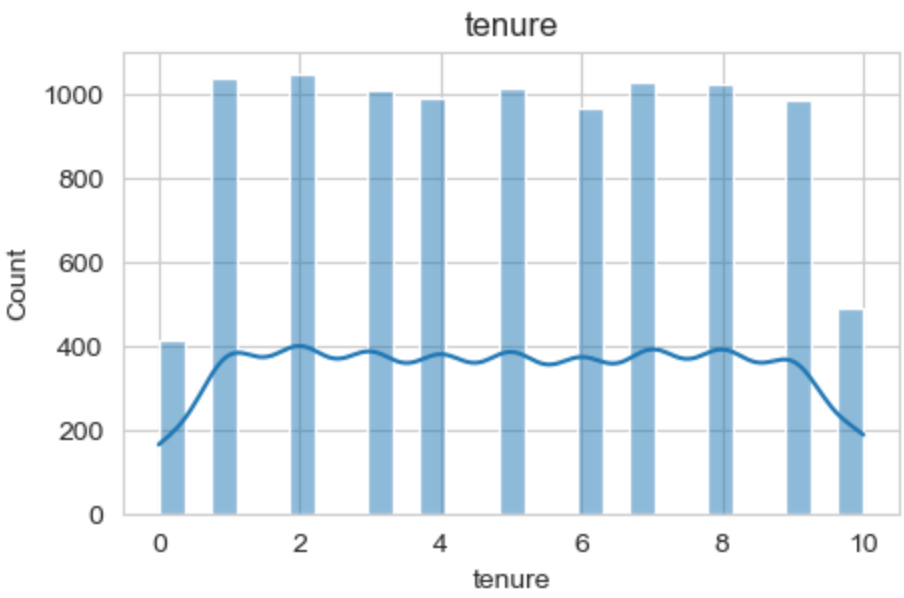


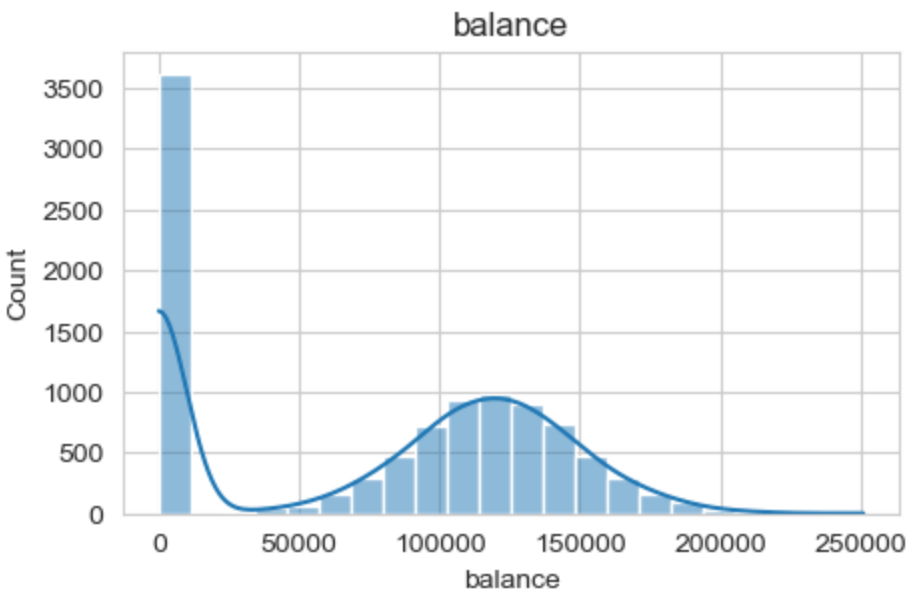
OBSERVATIONS:I used a loop to create count plots for all categorical variables against churn. This helps analyze how different categories impact customer churn and I found that country has stronger impact than gender.

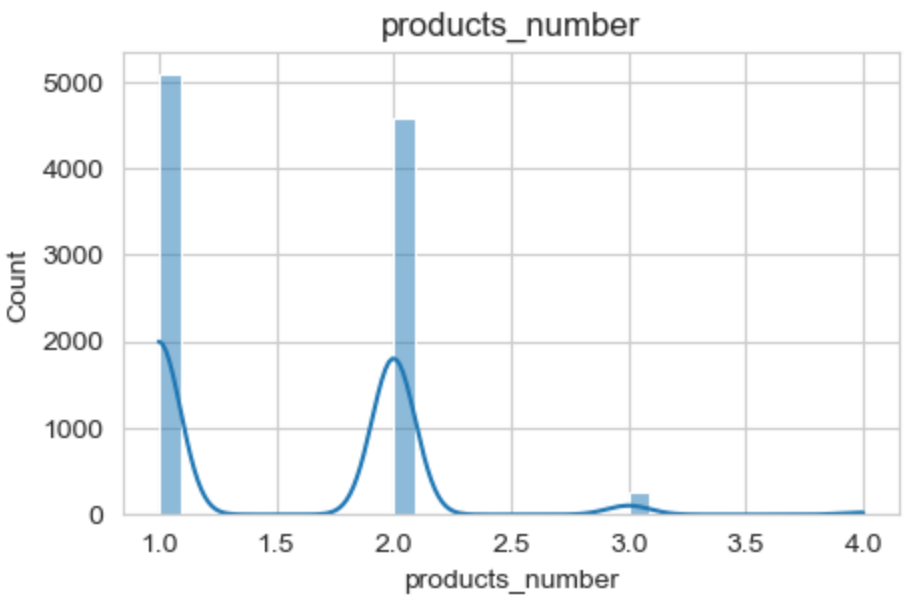
In [78]:
```
for col in num_cols:
   plt.figure(figsize=(5,3))
   sns.histplot(data[col], kde=True)
   plt.title(col)
   plt.show()
```


credit_score
Count
0
100
200
300
400
500
400
500
600
700
800
credit_score


age
Count
0
200
400
600
800
20
30
40
50
60
70
80
90
age


tenure
Count
1000
800
600
400
200
0
0
2
4
6
8
10
tenure


balance
Count
3500
3000
2500
2000
1500
1000
500
0
0
50000
100000
150000
200000
250000
balance



OBSERVATIONS:I plotted histograms with KDE for all numerical features to understand their distribution, skewness, and presence of outliers.

1. Credit Score Distribution almost normal (bell-shaped) Most values range is 600–700

2.Age Right-skewed (positive skew) Young customers are more Older customers are less

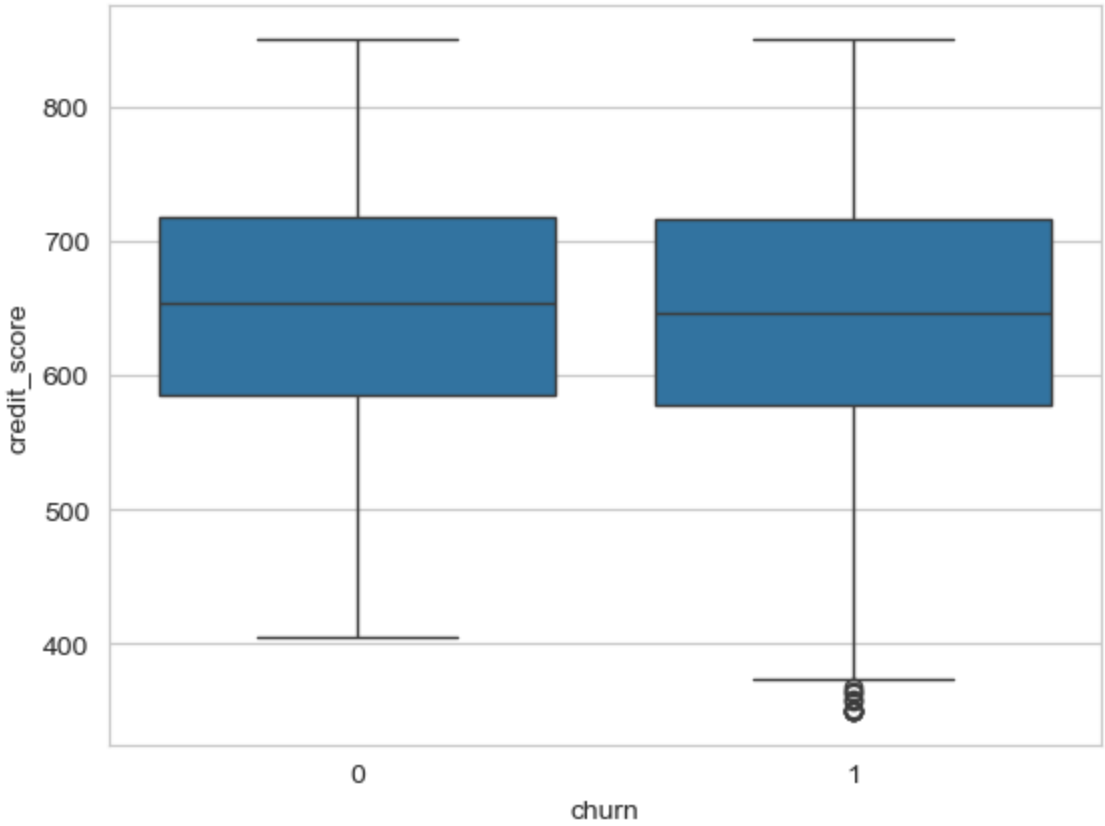
3. Tenure Uniform distribution (0–10 years) in every range customers are almost equal

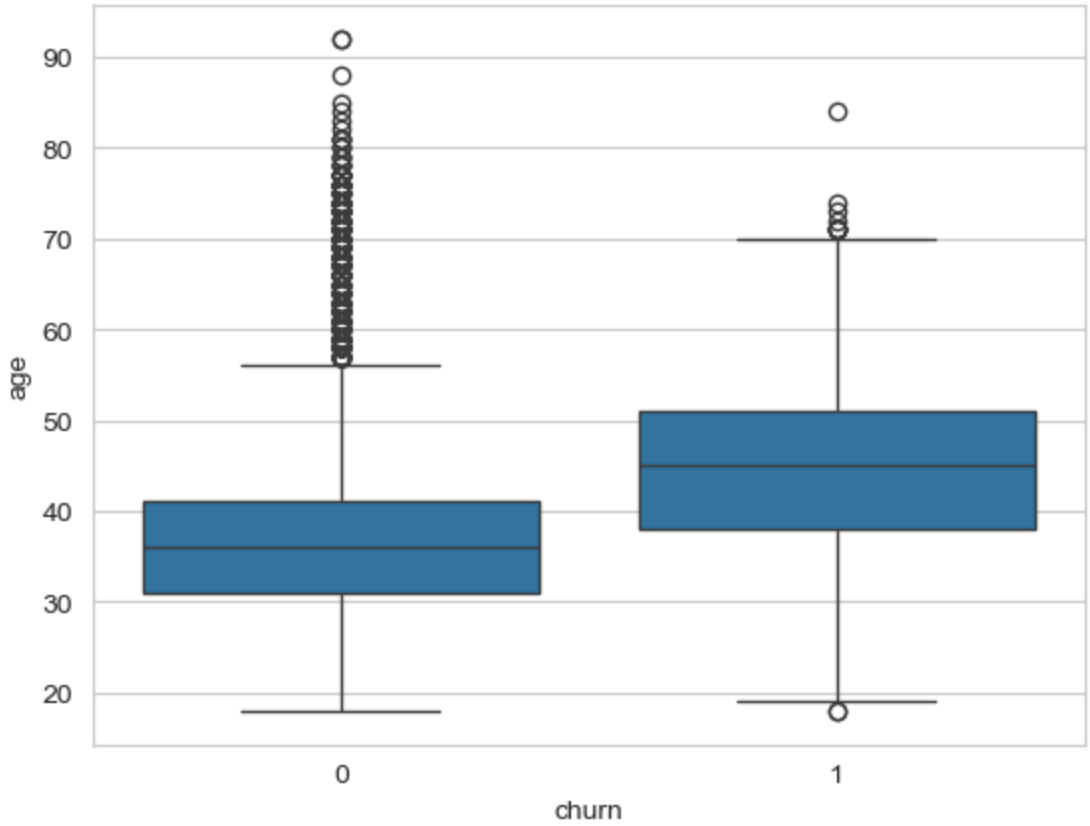
4. Balance Highly skewed many customers having zero balance

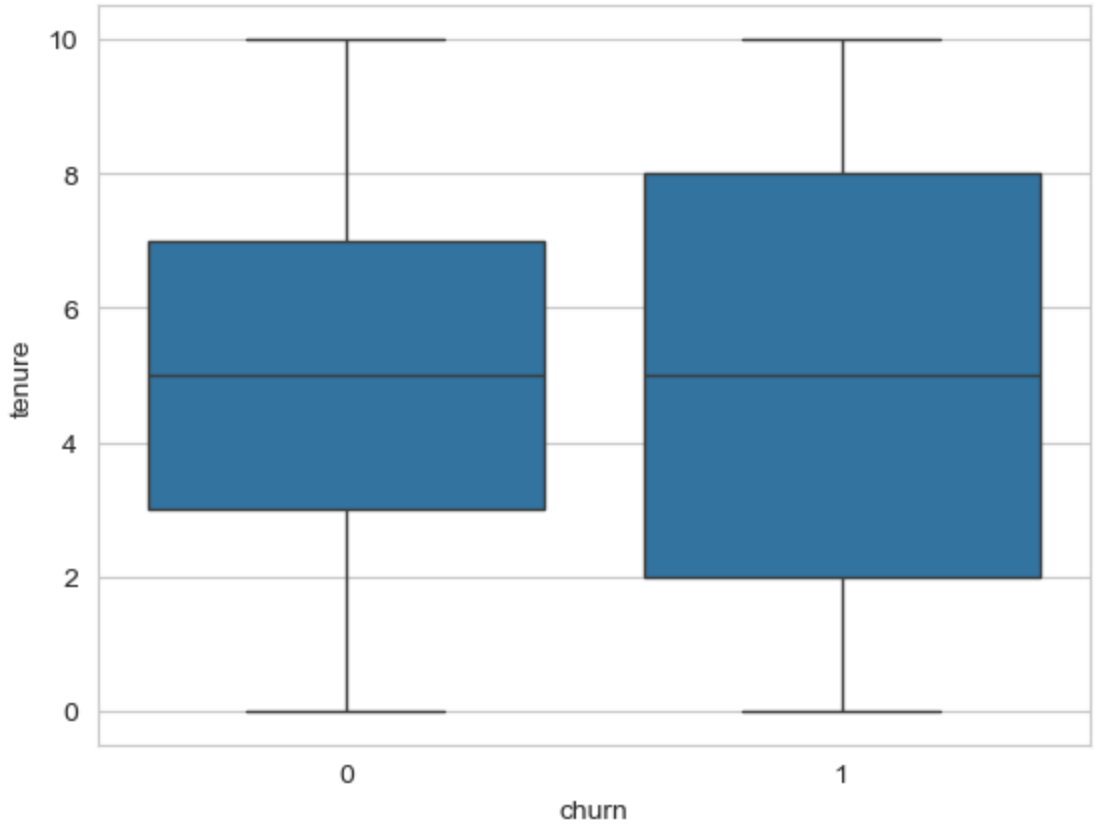
5. Products Number Discrete distribution Most customers → 1 or 2 products

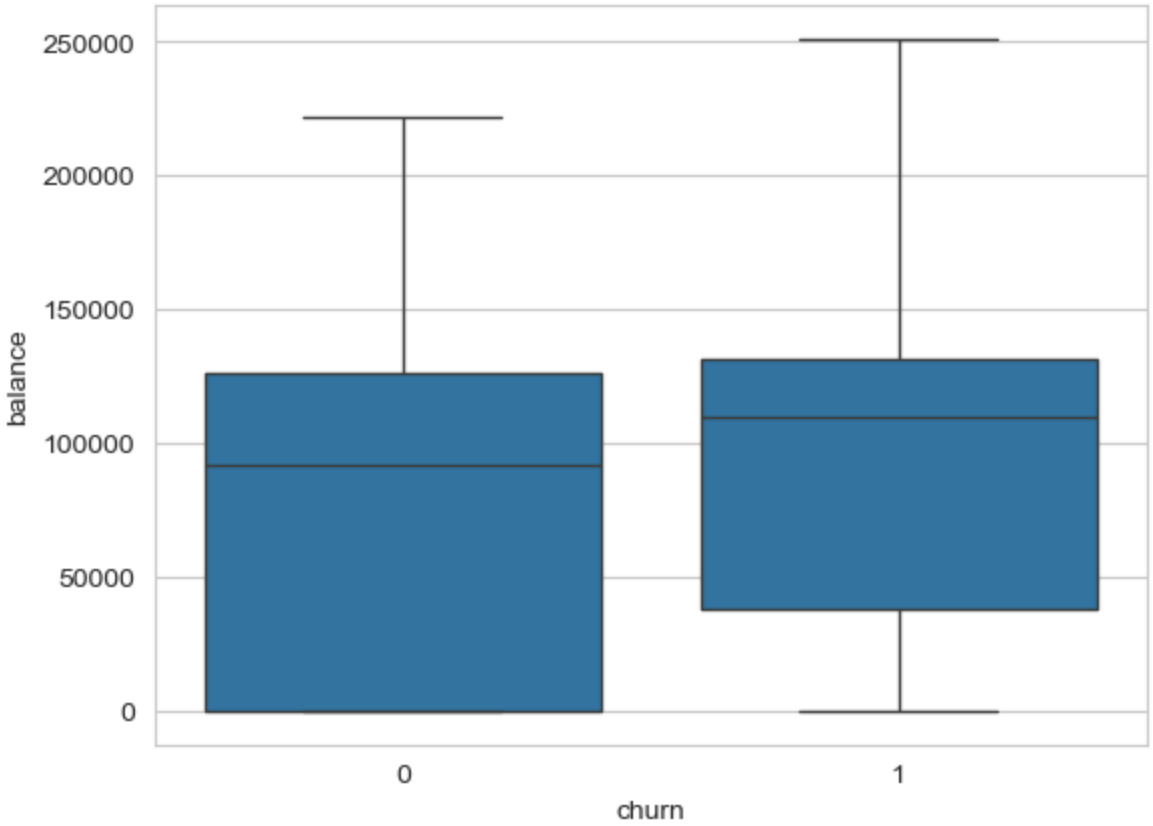
6. Estimated Salary Uniform distribution Salary spread evenly

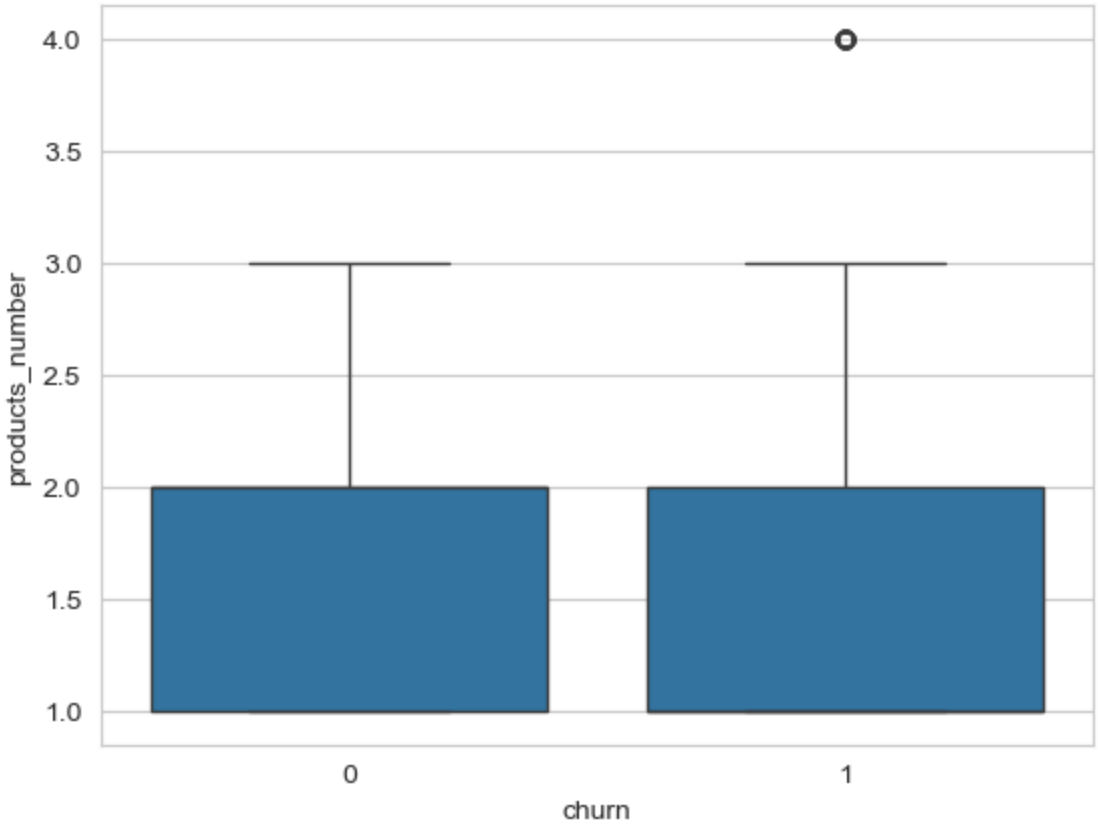
In [79]:
```
for col in num_cols:
    sns.boxplot(x='churn', y=col, data=data)
    plt.show()
```

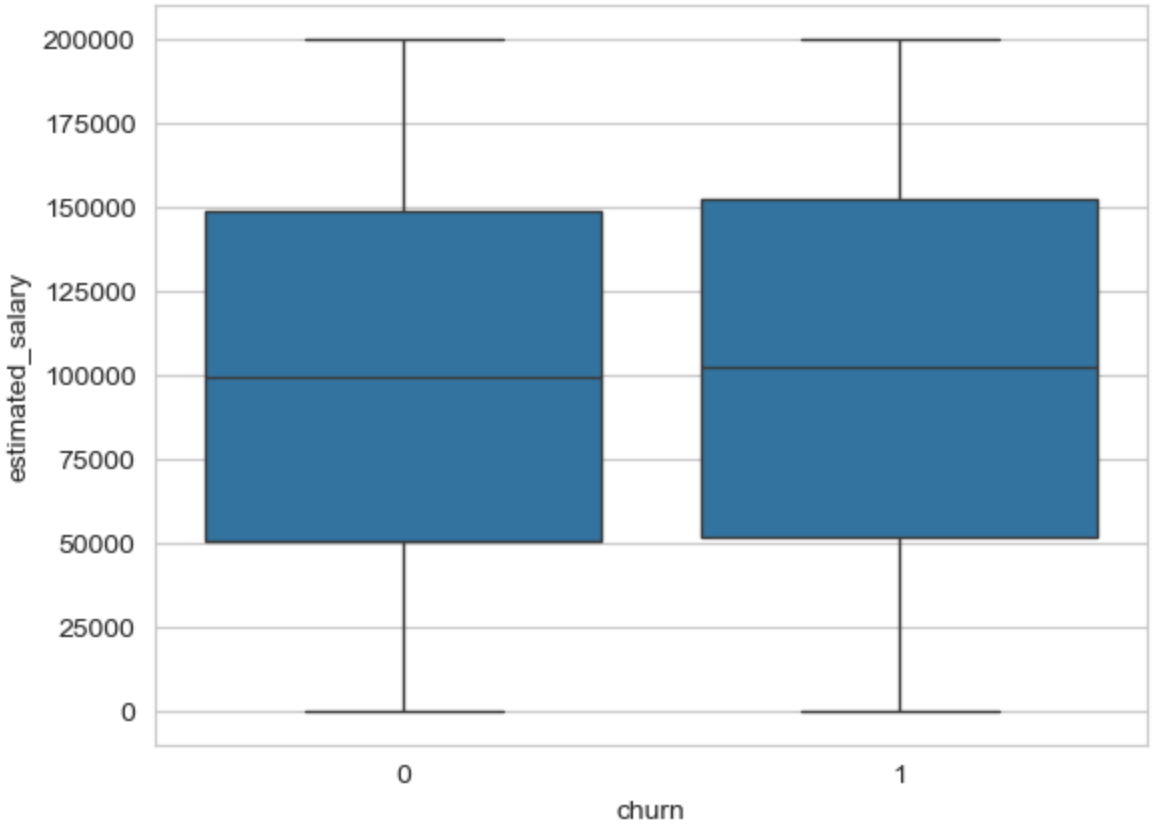

800
700
600
500
400
credit_score
0
1
churn


90
80
70
60
50
40
30
20
age
0
1
churn


10
8
6
4
2
0
tenure
0
1
churn


250000
200000
150000
100000
50000
0
balance
0
1
churn


4.0
3.5
3.0
2.5
2.0
1.5
1.0
products_number
0
1
churn


200000
175000
150000
125000
100000
75000
50000
25000
0
estimated_salary
0
1
churn

OBSERVATIONS:I used boxplots to compare numerical features across churn categories to identify differences in distributions and detect key patterns

1. Credit Score Churned customers having slightly lower score
2. Age Churned customers are older Its showing strong relationship with churn
3. Tenure not more difference Weak feature
4. Balance Churned customers haing higher balance Strong indicator
5. Products Number Most customers having one or two products Important feature
6. Estimated Salary Almost same distribution Weak impact

In [80]:
```
data.drop(columns=['customer_id'], inplace=True, errors='ignore')
data
```

Out[80]:

	credit_score	country	gender	age	tenure	balance	products_number	credit_card	active_member	estimated_salary
0	619	France	Female	42	2	0.00	1	1	1	101348.88
1	608	Spain	Female	41	1	83807.86	1	0	1	112542.58
2	502	France	Female	42	8	159660.80	3	1	0	113931.57
3	699	France	Female	39	1	0.00	2	0	0	93826.63
4	850	Spain	Female	43	2	125510.82	1	1	1	79084.10
...	...	...	...	...	...	...	...	...	...	...
9995	771	France	Male	39	5	0.00	2	1	0	96270.64
9996	516	France	Male	35	10	57369.61	1	1	1	101699.77
9997	709	France	Female	36	7	0.00	1	0	1	42085.58
9998	772	Germany	Male	42	3	75075.31	2	1	0	92888.52
9999	792	France	Female	28	4	130142.79	1	1	0	38190.78

10000 rows × 11 columns

OBSERVATIONS: I removed the customer_id column because it is just a unique identifier and does not contribute to predicting churn.

Age Binning (Feature Engineering)

In [81]:
```
print(data['age'].head())
print(data['age'].dtype)
```

```
0    42
1    41
2    42
3    39
4    43
Name: age, dtype: int64
int64
```

OBSERVATIONS:I checked sample values and datatype of the age column to ensure data consistency and confirm that it is a numerical feature.

In [82]:
```
print(data['age'].min(), data['age'].max())
```

```
18 92
```

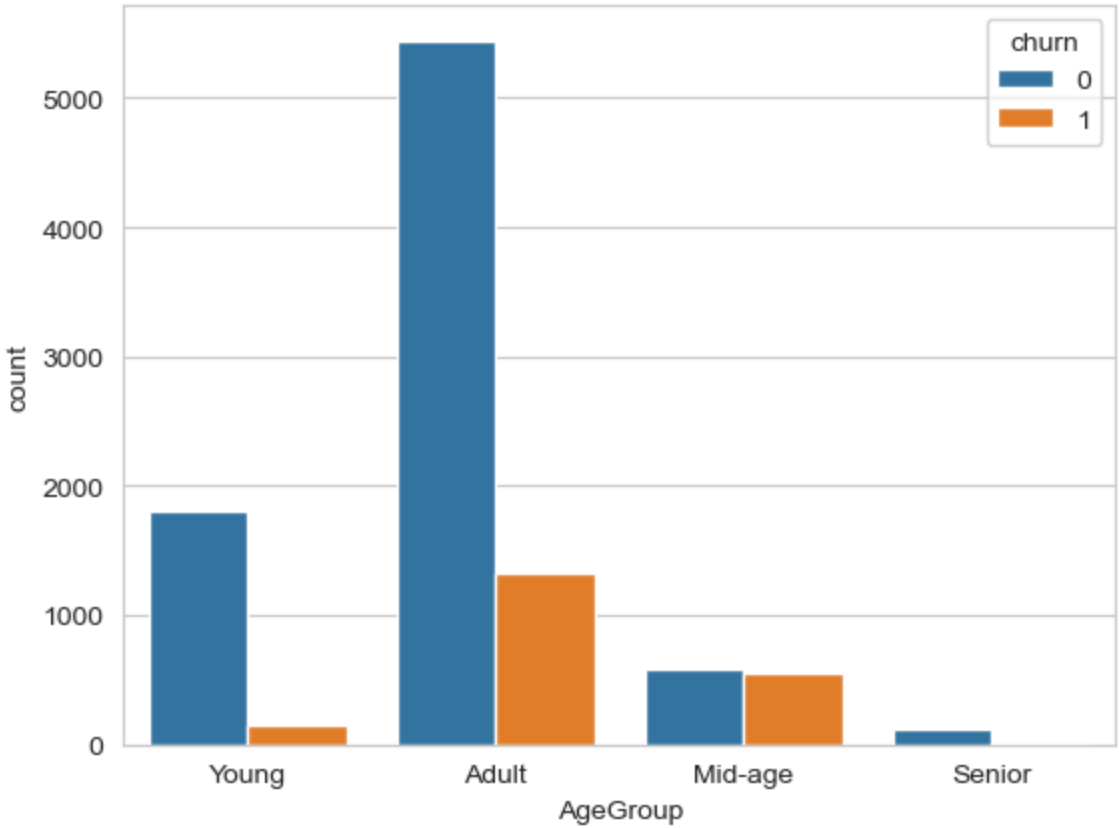
OBSERVATIONS:I checked the minimum and maximum values of the age column to ensure there are no unrealistic or incorrect values

In [83]:
```
data['AgeGroup'] = pd.cut(
    data['age'],
    bins=[18,30,50,70,100],
    labels=['Young','Adult','Mid-age','Senior']
)

print(data['AgeGroup'].value_counts())
```

```
AgeGroup
Adult      6771
Young      1946
Mid-age    1128
Senior      133
Name: count, dtype: int64
```

OBSERVATIONS:I created an AgeGroup feature by binning the age variable into meaningful categories to capture behavioral patterns across different age segments.

In [84]:
```
sns.countplot(x='AgeGroup', hue='churn', data=data)
plt.show()
```



OBSERVATIONS:I analyzed churn across different age groupS using countplot and found that churn increases with age, especially in mid-age and senior customers.

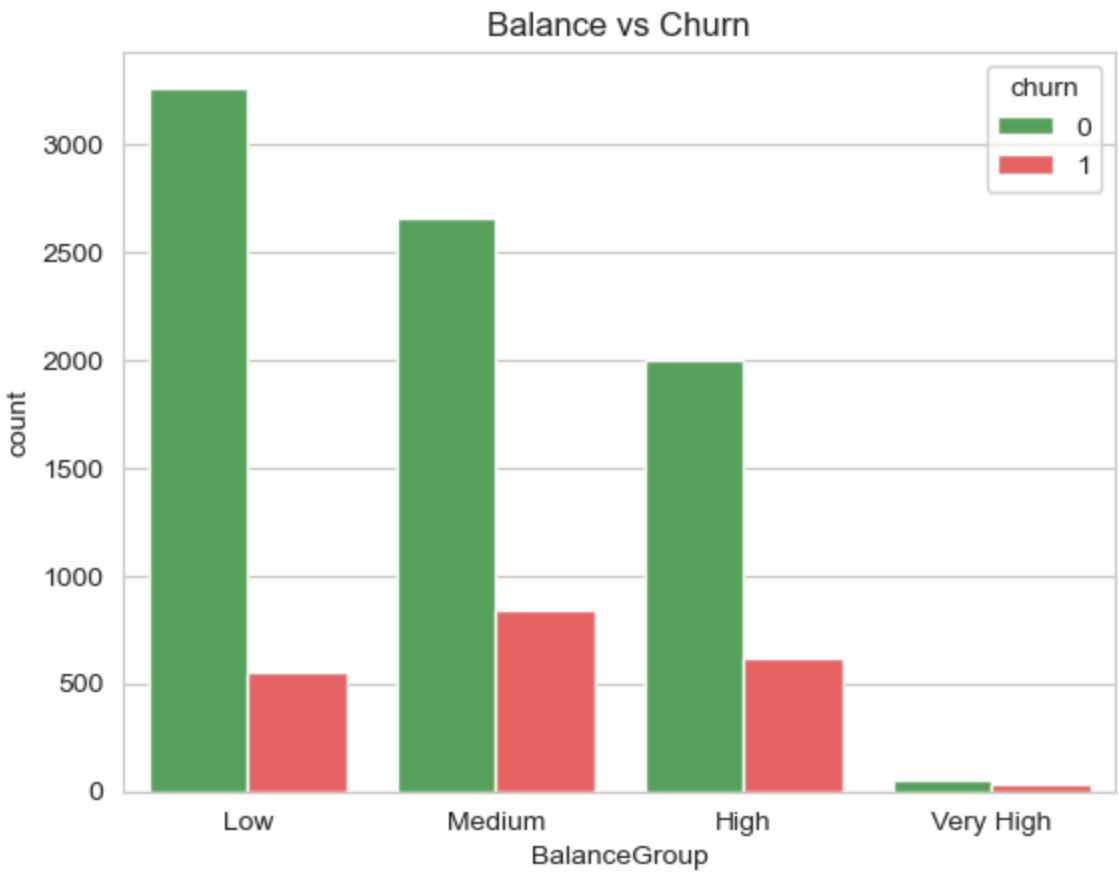
STEP 1: Balance Binning

In [85]:
```
data['BalanceGroup'] = pd.cut(
    data['balance'],
    bins=4,
    labels=['Low','Medium','High','Very High']
)

print(data['BalanceGroup'].value_counts())
```

```
BalanceGroup
Low          3813
Medium       3493
High         2612
Very High      82
Name: count, dtype: int64
```

OBSERVATIONS:Icreated BalanceGroup by binning the balance feature into four categories to better analyze customer segments based on account balance Most customers fall under the low balance category

In [86]:
```
sns.countplot(
    x='BalanceGroup',
    hue='churn',
    data=data,
    palette=['#4CAF50', '#FF5252']  # green & red
)

plt.title("Balance vs Churn")
plt.show()
```



OBSERVATIONS:I found that customers with higher account balances are more likely to churn, especially in the high and very high balance groups High balance customers are valuable, so their churn is a major business concern

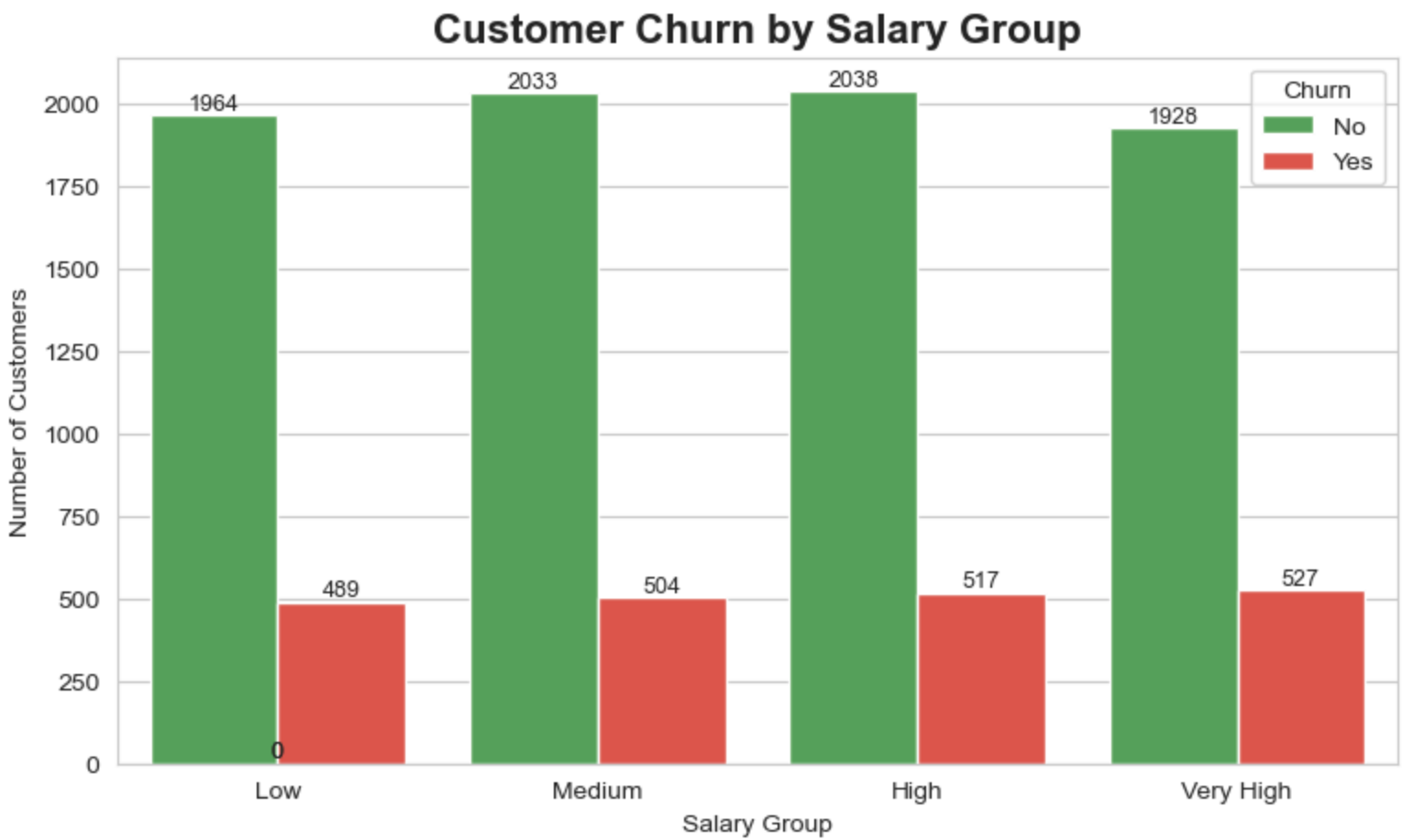
Salary Binning

In [87]:
```
data['SalaryGroup'] = pd.cut(
    data['estimated_salary'],
    bins=4,
    labels=['Low','Medium','High','Very High']
)

print(data['SalaryGroup'].value_counts())
```

```
SalaryGroup
High         2555
Medium       2537
Very High    2455
Low          2453
Name: count, dtype: int64
```

OBSERVATIONS:I created SalaryGroup by binning estimated salary into four categories to analyze customer segments based on income levels Salary does not show strong variation across customers

In [88]:
```
sns.set_style('whitegrid')

plt.figure(figsize=(9,5))

ax = sns.countplot(
    x='SalaryGroup',
    hue='churn',
    data=data,
    palette=['#4CAF50', '#F44336']
)

# Value labels
for p in ax.patches:
    ax.annotate(
        f'{int(p.get_height())}',
        (p.get_x() + p.get_width()/2., p.get_height()),
        ha='center', va='bottom', fontsize=9
    )

plt.title("Customer Churn by Salary Group", fontsize=16, fontweight='bold')
plt.xlabel("Salary Group")
plt.ylabel("Number of Customers")

plt.legend(title='Churn', labels=['No', 'Yes'])

plt.show()
```

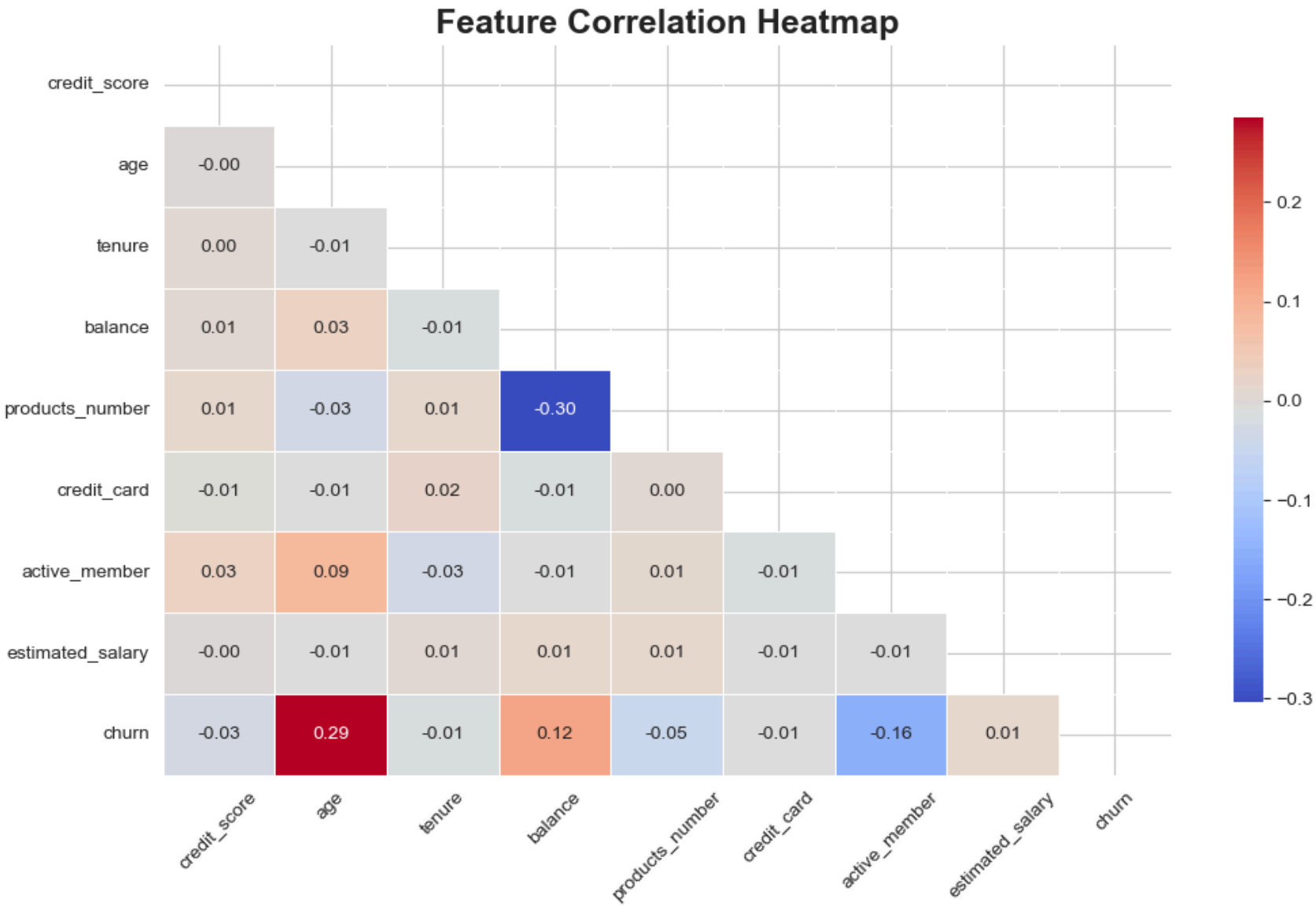


OBSERVATIONS:I analyzed churn across salary groups and found that churn distribution is fairly consistent across all salary levels, indicating that salary does not have a strong impact on churn

Correlation Heatmap

In [89]:
```
import numpy as np

corr = data.corr(numeric_only=True)
mask = np.triu(np.ones_like(corr, dtype=bool))

plt.figure(figsize=(12,7))
```

```
sns.heatmap(
    corr,
    mask=mask,
    annot=True,
    cmap='coolwarm',
    fmt='.2f',
    linewidths=0.5,
    cbar_kws={'shrink': 0.8}
)

plt.title("Feature Correlation Heatmap", fontsize=18, fontweight='bold')
plt.xticks(rotation=45)
plt.yticks(rotation=0)

plt.show()
```



OBSERVATIONS:I used a correlation heatmap to understand relationships between features. I found that age and balance have positive correlation with churn, while active membership has a negative correlation. Most other features show weak correlation, indicating they have less impact on churn.

Encoding i will convert categorical columns into numerical columns

In [90]:
```
data = pd.get_dummies(data, columns=['country', 'gender'], drop_first=True)
```

In [91]:
```
data
```

Out[91]:

	credit_score	age	tenure	balance	products_number	credit_card	active_member	estimated_salary	churn	AgeGroup
0	619	42	2	0.00	1	1	1	101348.88	1	Adult
1	608	41	1	83807.86	1	0	1	112542.58	0	Adult
2	502	42	8	159660.80	3	1	0	113931.57	1	Adult
3	699	39	1	0.00	2	0	0	93826.63	0	Adult
4	850	43	2	125510.82	1	1	1	79084.10	0	Adult
...	...	...	...	...	...	...	...	...	...	...
9995	771	39	5	0.00	2	1	0	96270.64	0	Adult
9996	516	35	10	57369.61	1	1	1	101699.77	0	Adult
9997	709	36	7	0.00	1	0	1	42085.58	1	Adult
9998	772	42	3	75075.31	2	1	0	92888.52	1	Adult
9999	792	28	4	130142.79	1	1	0	38190.78	0	Young

10000 rows × 15 columns

I applied one-hot encoding to categorical variables and used drop_first=True to avoid multicollinearity. I converted categorical data into numerical using one-hot encoding.Encoding ensures categorical features are properly used in machine learning models.

FEATURE SCALING

In [92]:
```
from sklearn.preprocessing import StandardScaler, MinMaxScaler       #  its very very important for real senerio
```

In [93]:
```
# Apply StandardScaler
standard_scaler = StandardScaler()
```

```
X_standard_scaled = standard_scaler.fit_transform(numeric_data)
X_standard_scaled
```

Out[93]:
```
array([[-0.32622142,  0.29351742, -1.04175968, -1.22584767, -0.91158349,
         0.02188649],
       [-0.44003595,  0.19816383, -1.38753759,  0.11735002, -0.91158349,
         0.21653375],
       [-1.53679418,  0.29351742,  1.03290776,  1.33305335,  2.52705662,
         0.2406869 ],
       ...,
       [ 0.60498839, -0.27860412,  0.68712986, -1.22584767, -0.91158349,
        -1.00864308],
       [ 1.25683526,  0.29351742, -0.69598177, -0.02260751,  0.80773656,
        -0.12523071],
       [ 1.46377078, -1.04143285, -0.35020386,  0.85996499, -0.91158349,
        -1.07636976]], shape=(10000, 6))
```

observations:I applied StandardScaler to normalize numerical features so that all variables are on the same scale, which helps in better comparison and prepares the data for future modeling

In [94]:
```
data_standard_scaled = pd.DataFrame(X_standard_scaled,columns=numeric_data.columns)
data_standard_scaled
```

Out[94]:

	credit_score	age	tenure	balance	products_number	estimated_salary
0	-0.326221	0.293517	-1.041760	-1.225848	-0.911583	0.021886
1	-0.440036	0.198164	-1.387538	0.117350	-0.911583	0.216534
2	-1.536794	0.293517	1.032908	1.333053	2.527057	0.240687
3	0.501521	0.007457	-1.387538	-1.225848	0.807737	-0.108918
4	2.063884	0.388871	-1.041760	0.785728	-0.911583	-0.365276
...	...	...	...	...	...	...
9995	1.246488	0.007457	-0.004426	-1.225848	0.807737	-0.066419
9996	-1.391939	-0.373958	1.724464	-0.306379	-0.911583	0.027988
9997	0.604988	-0.278604	0.687130	-1.225848	-0.911583	-1.008643
9998	1.256835	0.293517	-0.695982	-0.022608	0.807737	-0.125231
9999	1.463771	-1.041433	-0.350204	0.859965	-0.911583	-1.076370

10000 rows × 6 columns

OBSERVATIONS:After applying StandardScaler, the output was a NumPy array. So I converted it back into a DataFrame to retain column names and make the data easier to interpret

In [95]:
```
numeric_data.describe().T
```

Out[95]:

	count	mean	std	min	25%	50%	75%	max
credit_score	10000.0	650.528800	96.653299	350.00	584.00	652.000	718.0000	850.00
age	10000.0	38.921800	10.487806	18.00	32.00	37.000	44.0000	92.00
tenure	10000.0	5.012800	2.892174	0.00	3.00	5.000	7.0000	10.00
balance	10000.0	76485.889288	62397.405202	0.00	0.00	97198.540	127644.2400	250898.09
products_number	10000.0	1.530200	0.581654	1.00	1.00	1.000	2.0000	4.00
estimated_salary	10000.0	100090.239881	57510.492818	11.58	51002.11	100193.915	149388.2475	199992.48

In [96]:
```
numeric_data.corr()
```

Out[96]:

	credit_score	age	tenure	balance	products_number	estimated_salary
credit_score	1.000000	-0.003965	0.000842	0.006268	0.012238	-0.001384
age	-0.003965	1.000000	-0.009997	0.028308	-0.030680	-0.007201
tenure	0.000842	-0.009997	1.000000	-0.012254	0.013444	0.007784
balance	0.006268	0.028308	-0.012254	1.000000	-0.304180	0.012797
products_number	0.012238	-0.030680	0.013444	-0.304180	1.000000	0.014204
estimated_salary	-0.001384	-0.007201	0.007784	0.012797	0.014204	1.000000

OBSERVATIONS:Age shows positive correlation with churn Balance has weak correlation with churn Most features have low correlation, indicating weak linear relationships

KEY INSIGHTS

1. Overall Churn Situation Around 20% customers left the bank Simple meaning: Out of every 5 customers, 1 customer is leaving Insight: This is not very high, but still a serious business problem Company needs to focus on customer retention

2.Age is a Strong Factor Older customers have higher churn Simple meaning: Young people stay, older people leave more Why this happens: Older customers compare banks more They expect better service They may shift for better offers

3.High Balance Customers Are Leaving Customers with more money in account having more churn Simple meaning: Rich customers are leaving more Why this is important: These are valuable customers Losing them = big financial loss

4.Active Member is the MOST IMPORTANT Factor Inactive customers churn a lot Active customers rarely leave Simple meaning: those customers are not using the bank servuces they are leaving the bank

5. Country One country has higher churn Simple meaning: customer behaviour is different in every country Why this happens: Competition Service quality Local market conditions

6. Gender Has Very Little Impact Male & Female churn almost same Simple meaning: Gender doesn't matter much here

7. Credit Card Has No Major Impact

8. Correlation Insight

Most values are close to 0 Simple meaning: Features are not strongly connected Insight: No strong linear relationship No multicollinearity problem

Deep meaning: Churn depends on multiple factors together, not one single factor

conculsion

From the analysis, I found that customer churn is mainly influenced by customer engagement, age, and account balance. The most important factor is whether the customer is active or not, as inactive customers are much more likely to leave. Additionally, older customers and those with higher account balances show higher churn, indicating dissatisfaction among valuable customers. On the other hand, features like gender, salary, and credit card ownership have minimal impact. Overall, the key driver of churn is lack of engagement, and improving customer interaction can significantly reduce churn

RECOMMENDATIONS:

1. Target inactive users with special offers.
2. Focus on Germany – run Germany-specific retention campaigns.
3. Increase product cross selling (1 product → 2+ products).

4. Implement loyalty programs for high balance customers.
5. Provide extra support to first-year customers (calls, offers, check-ins).

In [ ]:

```

```